

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ - ĐHQGHN

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

G3_BIDDING_ONLINE

Đề tài: Hệ thống đấu giá trực tuyến

Lớp học phần: UET.CS2043 3

Giảng viên: TS. Nguyễn Văn Sơn

**Phụ trách Thực hành: CN. Phạm Bảo Phúc
CN. Trần Ngọc Trúc Linh**

Nhóm	3
Thành viên	Trương Thành Đạt – 25020110 Nguyễn Việt Anh - 25020024 Nguyễn Bá Thái Dương – 25020095 Đặng Tiến Đạt – 25020103
Repository	https://github.com/trdat19/G3_Bidding_Online
Video demo	Link video đầy đủ trong README

Hà Nội, 31 tháng 5 năm 2026

PHẦN 1: GIỚI THIỆU MỤC TIÊU VÀ PHẠM VI ĐỀ TÀI

1.1 – Mục tiêu

- Xây dựng hệ thống đấu giá trực tuyến cho phép nhiều người dùng tham gia cạnh tranh đấu giá sản phẩm hay dịch vụ theo thời gian thực. Hệ thống hỗ trợ 3 vai trò chính: Admin, Seller, Bidder với các chức năng cụ thể cho từng vai trò. Áp dụng lập trình hướng đối tượng vào hệ thống, JavaFX, Socket, mô hình Client-Server, MVC, xử lý đồng thời và kiểm thử tự động.

1.2 – Phạm vi

- Hệ thống gồm các chức năng chính: Đăng kí/Đăng nhập, Quản lý sản phẩm, Quản lý người dùng, Thêm sản phẩm, Tạo phiên đấu giá, Tham gia đấu giá, Cập nhật Realtime, Tự động kết thúc phiên, Xác định người thắng, Xem lịch sử sản phẩm đã thắng, và các chức năng nâng cao được triển khai tùy mức hoàn thiện của nhóm.

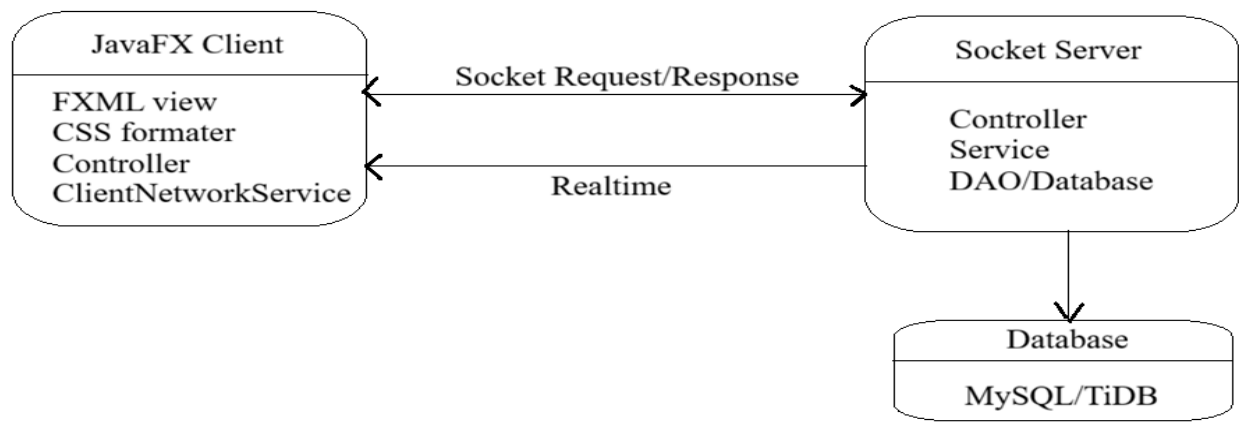
PHẦN 2: CÁC CHỨC NĂNG CHÍNH ĐÃ HOÀN THÀNH THEO BAREM

Nội dung đánh giá	Nội dung đã hoàn thành	Hướng giải quyết, lý do lựa chọn	Trạng thái
Xây dựng các lớp chính: người dùng, sản phẩm	Các lớp chính trong model: như User, Bid, Auction, AutoBidRule	Mỗi lớp biểu diễn một thực thể hoặc vai trò cụ thể trong hệ thống. Cách tổ chức giúp mã nguồn bám sát nghiệp vụ đấu giá và dễ mở rộng.	Hoàn thành
Các chức năng chính	Seller đăng sản phẩm, gửi yêu cầu tạo phiên đấu giá, Admin chấp nhận yêu cầu, khoá-mở khoá tài khoản người dùng, Bidder tham gia đặt giá trực tiếp	Cơ sở dữ liệu lưu thông tin quan trọng, phục vụ cho hệ thống hoạt động hoàn chỉnh. DAOs cho phép server truy cập dữ liệu. Controller gọi các Service để thực hiện nghiệp vụ. Xây dựng theo mô hình Client-Server, MVC để quan sát	Hoàn thành
Concurrency	Có lớp quản lý khoá chung của Server. Cấp khoá chung cho các logic nghiệp vụ quan trọng của hệ thống	Hệ thống sử dụng ReentrantLock theo từng auctionId để đồng bộ các thao tác quan trọng trong cùng một phiên đấu giá. Khi một thread đang xử lý đặt giá cho một phiên, các thread khác muốn cập nhật cùng phiên đó phải chờ lock được giải phóng.	Hoàn thành
Realtime	RealtimePushServer, Listener	Loa thông báo, Observer qua Socket, Client cập nhật UI bằng Platform.runLater() giúp các client hiển thị được ngay những event mới xảy ra	Hoàn thành
Lỗi/ngoại lệ	Các lỗi, dữ liệu không hợp lệ liên quan tới phiên đấu giá: Giá quá thấp, không hợp lệ, phiên đã đóng, ...	Có custom exception riêng với lỗi cụ thể.	Hoàn thành
Tích hợp, kiến trúc, chất lượng mã	Đã có kiến trúc Client-Server rõ ràng, áp dụng MVC. Sử dụng Maven thiết lập CI/CD tự động test khi	Mô hình Client-Server dễ quan sát và bảo trì. Sử dụng Maven đơn giản dễ dàng hơn với những dự án nhỏ không quá phức tạp. Junit kết hợp với Mockito hỗ trợ xây dựng bộ test với	Hoàn thành

	push lên repo	DAOs, Service có Singleton Pattern mà không phải kết nối Database thật.	
Nâng cao	Auto-bidding, Anti-sniping, Bid History LineChart	PriorityQueue, ReentrantLock theo auctionId, các thao tác xử lý dùng chung lock giúp tránh race condition, lost update	Anti-sniping đã hoàn thiện, Auto-bidding và Bid History LineChart được hoàn thiện ở mức cơ bản

PHẦN 3: KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG

- Hệ thống tổ chức theo mô hình Client-Server. Client JavaFX cung cấp giao diện người dùng, gửi đi request và nhận response/event từ server. Server nhận request chịu trách nhiệm kiểm tra nghiệp vụ, xử lý đồng thời, truy cập lấy dữ liệu từ database, phát thông báo realtime. Database chỉ được truy cập duy nhất từ server nhằm nhất quán dữ liệu.

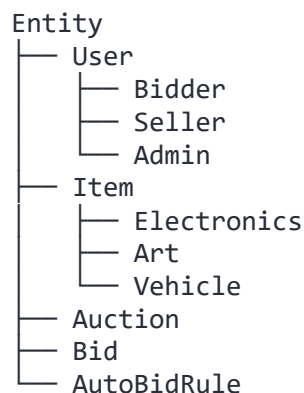


PHẦN 4: THIẾT KẾ MVC VÀ CẤU TRÚC CỦA HỆ THỐNG

Tầng	Thành phần tiêu biểu	Vai trò
Client - View	FXML, CSS	Hiển thị màn hình đăng nhập, dashboard của các vai trò, màn hình đấu giá,...
Client – Controller	LoginController, BiddingViewController,...	Nhận sự kiện từ người dùng, ClientNetworkService gửi đi request
Shared - DTO	BaseRequest, BaseResponse, DTOs	Ngôn ngữ chung của dữ liệu truyền qua Socket giữa Client và Server. DTOs dữ liệu được gói từ Server gửi lên Client.
Server – RequestRouter	RequestRouter	Từ sự kiện nhận được từ Client, điều phối gọi các Controller tương ứng

Server – Controller	AuthServerController, AuctionServerController, BidServerController	Gọi đến các service tương ứng.
Server – Service	AuctionService, BidService, AuthService	Xử lý nghiệp vụ kèm kiểm tra hợp lệ, đảm bảo đa luồng
Server - DAO	UserDAO, BidDAO, AuctionDAO	Truy cập DB, map dữ liệu thành model

PHẦN 5: THIẾT KẾ HƯỚNG ĐỐI TƯỢNG, DESIGN PATTERN



Các nguyên tắc OOP được áp dụng như sau: Encapsulation thông qua thuộc tính private/protected và getter/setter; Inheritance trong phân cấp User và Item; Polymorphism khi các role hoặc item override thông tin riêng; Abstraction bằng lớp cha/interface cho các thực thể chung.

Pattern	Vị trí áp dụng	Lý do
Singleton	Services, AuctionScheduler, DBConnection, AuctionLockManager	Dùng 1 instance duy nhất quản lý tài nguyên chung.
Factory Method, Abstract Factory	Item Factory	Tạo object theo loại sản phẩm, giảm phụ thuộc class cụ thể
Observer	RealtimePushServer, RequestRouter, ClientNetworkService, controller,.	Có nhiều client xem phiên. Khi có thay đổi mới, server phát event khi đó các client đang theo dõi sẽ tự nhận và cập nhật UI
DAO	UserDAO, AuctionDAO, BidDAO, ItemDAO, AutoBidRuleDAO	Truy vấn Database
DTO	BaseRequest, BaseResponse, DTOs	Chuẩn hoá thông điệp chung Client – Server

PHẦN 6: THIẾT KẾ CƠ SỞ DỮ LIỆU

Bảng	Trường chính	Mục đích
users	id, username, password, full_name, email, role, status, created_at	Quản lý tài khoản, phân quyền
items	id_item, name_item, category, description, id_seller,	Thông tin của sản phẩm

	status_item, created_atItem, image_url	
auctions	id_auction, id_item, id_seller, start_price, max_price, min_increment, buy_now_price, start_time, end_time, status_auction	Quản lý phiên đấu giá
bids	id, auction_id, bidder_id, bid_amount, bid_time,	Lưu lịch sử đặt giá
auto_bid_rules	id, auction_id, bidder_id, max_amount, step_amount, is_active, created_at, updated_at	Cấu hình của 1 lệnh đấu giá tự động

PHẦN 7: LƯỒNG XỬ LÝ NGHIỆP VỤ CỦA HỆ THỐNG

*Bidder nhập giá

- > Client nhận dữ liệu tạo BaseRequest (action = PLACE_BID, Map data = auctionId + amount) và gửi đi bằng ClientNetworkService
- > Server RequestRouter nhận action điều phối gọi BidServerController để gọi nghiệp vụ
- > BidService xử lý lệnh đặt giá, lấy khóa theo auctionId
- > Kiểm tra phiên còn RUNNING, giá hợp lệ và không thấp hơn minIncrement
- > BidDAO insert bid, AuctionDAO cập nhật current_price/winner
- > AutoBidService.reactToIncomingBid phản ứng nếu có rule phù hợp.
- > RealtimePushService gửi event cho các client đang xem phiên
- > Client cập nhật giao diện realtime

PHẦN 8: KHẢ NĂNG NỔI BẬT

8.1 – Xử lý đấu giá đồng thời

- Vấn đề chính khi nhiều Bidder đặt giá cùng lúc là lost update, race condition, rollback giá hoặc hai người cùng thắng. Để xử lý, hệ thống cung cấp khóa riêng dùng theo từng auctionId. Như vậy các phiên khác nhau vẫn có thể xử lý song song, nhưng trong cùng một phiên chỉ có một thread được phép đọc giá hiện tại, kiểm tra hợp lệ, ghi bid và cập nhật winner tại một thời điểm.

8.2 – Auto-bidding

- Người dùng đặt Rule gồm maxAmount và stepAmount và bật rule. Khi có bid mới từ đối thủ, AutoBidService lấy các luật còn hiệu lực, đưa vào PriorityQueue sắp xếp theo độ ưu tiên maxAmount lớn hơn -> created_at sớm hơn -> id nhỏ hơn phù hợp với tiêu chí trong barem chấm điểm, và xác định kết quả cuối cùng và đặt bid cuối cùng; không insert các bid trung gian. PriorityQueue sắp xếp theo tiêu chí như vậy giúp xử lý được các rule auto-bid đồng thời, tránh xung đột

```
PriorityQueue<AutoBidRule> priorityQueue = new PriorityQueue<>(  
    Comparator.comparing(  
        AutoBidRule::getMaxAmount,  
        Comparator.reverseOrder())  
    ).thenComparing(  
        AutoBidRule::getCreatedAt,  
        Comparator.nullsLast(Comparator.naturalOrder())  
    ).thenComparing(  
        AutoBidRule::getId,
```

Comparator.**nullsLast**(Comparator.**naturalOrder**()))
);

8.3 – BidHistory – linechart

- Biểu đồ hiển thị mức giá đặt giá theo thời gian được cập nhật, hiển thị trực tiếp. Giúp Bidder theo dõi được tổng quan mức giá của phiên đấu giá đang diễn ra

8.4 – Anti-sniping.

- Đặt thời gian trong 60s cuối nhận bid mới, tiếp tục gia hạn phiên thêm 60s. Dùng `LocalDateTime.plusSecond(extraSecond)` và `setEndTime` lại cho auction. Push event tới cho client.

PHẦN 9: XỬ LÝ NGOẠI LỆ, REALTIME UPDATE

- Realtime update triển khai theo hướng event-based, tức gửi lên thông báo mỗi khi có sự kiện xảy ra, không polling liên tục. Khi có `PLACE_BID/Auction Finished`, server gửi event đến các `ClientConnectionHandler` đang subscribe trong `auctionId` tương ứng. Ở client, dữ liệu nhận được được đẩy về `JavaFX Application Thread` bằng `Platform.runLater()` để cập nhật bảng giá, người dẫn đầu và biểu đồ nếu có.

Tình huống lỗi	Phát hiện	Phản hồi cho client
Đặt giá thấp hơn yêu cầu	<code>amount < currentPrice + minIncrement</code>	<code>InvalidBidException</code>
Không tìm thấy phiên	<code>AuctionDAO.findById()</code> trả null	<code>AuctionNotFoundException</code>
Phiên đã đóng, quá thời gian	<code>auction.getStatus != RUNNING</code> , quá <code>endTime</code>	<code>AuctionClosedException</code>
Lỗi kết nối/DB error	<code>IOException/SQLException</code>	Server báo lỗi, trả <code>BaseResponse(false,...)</code>

PHẦN 10: KIỂM THỬ TỰ ĐỘNG VÀ ĐÁNH GIÁ

Hạng mục	Nội dung kiểm thử/triển khai	Kết quả mong muốn
JUnit, Mockito	Test <code>AuthService</code> , <code>BidService</code> , <code>AuctionService</code> với các trường hợp hợp lệ và không hợp lệ	Các test pass, service trả đúng kết quả hoặc ném đúng exception
Concurrency	Các client cùng gọi <code>placeBid()</code> vào cùng 1 phiên	Không lost update, rollback. Vẫn cập nhật chính xác giá hiện tại và xác định được winner
E2E demo	Chạy server và nhiều client bằng file JAR	Realtime hoạt động, các client đều nhận và hiển thị được event mới
CI/CD	Github Action chạy mvn test khi push lên repository	Gắn được badge passing vào file README trước deadline
Đóng gói	Maven-shade-plugin tạo fat JAR	Chạy được bằng lệnh <code>java -jar</code>

PHẦN 11: ĐÓNG GÓP CỦA CÁC THÀNH VIÊN

Thành viên	Module chính	Công việc cụ thể	Đóng góp
Trương Thành Đạt	<code>server.service</code> <code>server.scheduler</code> <code>server.model</code> <code>server.controller</code> <code>server.concurrency</code>	- <code>server.model</code> : bước đầu tạo mô hình nghiệp vụ chung, cốt lõi của hệ thống - <code>server.scheduler</code> : tạo scheduler, tự động đóng mở phiên khi tới thời gian - <code>server.concurrency</code> : tạo trình quản lý khoá hệ	25%

	Hoàn thành, báo cáo hệ thống	thống, cung cấp khoá riêng theo auctionId xử lý các logic nghiệp vụ quan trọng - server.service: tạo khung ban đầu cho các lớp xử lý nghiệp vụ - Thống nhất, làm sạch code, viết README, báo cáo, làm video demo.	
Nguyễn Việt Anh	Javafx: fxml, css client.controller client.service client.model	- Javafx: dùng các file .fxml để thiết kế màn hình, các file .css để định dạng giao diện. - client.Controller: chứa các controller của javafx, có nhiệm vụ xử lý sự kiện giao diện, gọi service để gửi yêu cầu lên server và cập nhật lại UI - client.service: Kết nối socket với server, gửi BaseRequest, nhận BaseResponse và lắng nghe realtime event - client.Model: Chứa các model dùng riêng cho giao diện client.	25%
Nguyễn Bá Thái Dương	server.network server.controller server.service	- server.network : Nhận kết nối từ client, đọc, điều phối request và gửi response - server.controller : Nhận request, bóc tách dữ liệu, gọi service và đóng gói kết quả - server.service : Logic của hệ thống, đối chiếu dữ liệu với Database	25%
Đặng Tiến Đạt	server.database server.dao server.model test/java/server	- server.database : kết nối tới Database cloud , test connection Database - server.dao : truy vấn và chỉnh sửa bảng liên quan tới phần đấu giá (AuctionDAO, UserDAO ,ItemDAO, BidDAO) - server.model : kiểm tra và chỉnh sửa các model (User , Item , Core) - test/java/server.service : thiết kế và chỉnh sửa và fixbug test - test/java/server.network : test request từ client điều đến controller xử lý	25%

- Các thành viên đồng thời hỗ trợ nhau trong các phần việc.

PHẦN 12: KẾT LUẬN

- Nhóm đã xây dựng hệ thống đấu giá trực tuyến theo kiến trúc Client-Server, hỗ trợ quản lý người dùng, sản phẩm, phiên đấu giá, đặt giá realtime, tự động kết thúc phiên và xử lý các tình huống lỗi. Hệ thống áp dụng OOP, MVC, các design pattern như Singleton, Factory Method, Observer, đồng thời sử dụng ReentrantLock để bảo đảm an toàn khi nhiều người dùng đấu giá đồng thời. Các chức năng nâng cao như Auto-Bidding và Anti-Sniping giúp hệ thống gần với nghiệp vụ thực tế hơn. Trong quá trình nộp bài, nhóm đã bảo đảm nhánh main là bản cuối, README đầy đủ, có executable JAR, báo cáo PDF tối đa 6 trang và video demo tối đa 3 phút.